MTH251 2023S1 Exam

Feel free to contact me at maths_and_programming@outlook.sg if you need tuition. I provide university level tuition for the following SUSS units:

* ENG201 Linear Systems Analysis and Design
* ENG301 Microprocessor Programming
* ICT101 Computational Thinking
* ICT133 Structured Programming
* ICT162 Object Oriented Programming
* MTD215 Application of C++ in Multimedia
* ICT231 Data Structures and Algorithms I
* ICT232 Data Structures and Algorithms II
* MTH105 Fundamentals of Mathematics
* MTH107 Calculus I
* MTH108 Calculus II
* MTH316 Multivariable Calculus
* MTH319 Mathematical Methods I
* MTH320 Mathematical Methods II
* MTH207 Linear Algebra
* MTH219 Fundamentals of Statistics and Probability
* MTH220 Statistical Methods and Inference
* MTH355 Basic Mathematical Optimisation

Donaldson Tan

SINGAPORE UNIVERSITY OF SOCIAL SCIENCES

MTH251

Examination - January Semester 2023

Data Structures and Algorithms I

Monday, 15 May 2023 **01:00 pm - 03:00 pm**

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This examination contains **SEVEN (7)** questions and comprises **SEVEN (7)** printed pages (including cover page).
2. You must answer **ALL** questions.
3. All answers must be written in the answer book. Marks will only be awarded if FULL working is shown.
4. This is **Open-book** examination.

At the end of the examination.
Please ensure that you have written your examination number on each answer book used.

Failure to do so will mean that your work cannot be identified.
If you have used more than one answer book, please tie them together with the string provided.

THE UNIVERSITY RESERVES THE RIGHT NOT TO MARK YOUR SCRIPT IF YOU FAIL TO FOLLOW THESE INSTRUCTIONS.

MTH251 Copyright © 2023 Singapore University of Social Sciences (SUSS)

Answer all questions. (Total 100 marks)

Question 1a

Order the following functions by their asymptotic growth rates in ascending order:

$$2nlog(n) + 4n,\ 2^{12},\ 5n + 2log(n),\ 3n^2.$$

Provide clear explanation.

(5 marks)

Question 1b

Analyze the time complexity of the following algorithm that return a list of factors of a given number.

```
def find_factors (num):
    result = []
    for i in range(1, num+1):
        if num%i == 0:
            result.append(i)
    return result
```

(5 marks)

Question 1c

Improve the algorithm in Question 1b such that it has a better time complexity.

(5 marks)

Question 2

Implement an algorithm in Python to solve the problem of determining whether a string s is a palindrome, i.e., s is the same when read forward or backward. For example, given 'ilovepython', the algorithm returns False; given 'ilovepythonohtypevoli', the algorithm returns True.

(10 marks)

Question 3a

Suppose that you need to develop an app that needs to store the addresses of recently visited sites with the condition that the latest address should be retrieved first. Among stack, queue and tree, which data type would you choose to store the addresses? Discuss your reason.

(2 marks)

Question 3b

Suppose that you need to develop an app that needs to store food orders from customers with the condition that early orders should be served first. Among stack, queue and tree, which data type would you choose to store the orders? Discuss your reason.

(3 marks)

Question 3c

Interpret the following code and write its output, where list_2 is intended to be independent of list_1.

```
a = [1,1,1]
b = [2,2,2]
c = [3,3,3]
list_1 = [a,b,c]
list_2 = list_1.copy()
list_2[0][0] = 0
print(list_1)
```

(5 marks)

Question 3d

Identify if the copy operation in Question 3c has bugs. If yes, correct the code.

(5 marks)

Question 4a

Given that my_stack is an empty instance of the Stack ADT. The following operations are performed. Interpret the operations and write down the elements in my_stack.

```
my_stack.push(10);
my_stack.push(12);
my_stack.pop();
my_stack.size();
my_stack.push(5);
my_stack.pop();
my_stack.push(14);
my_stack.push(10);
```

(5 marks)

Question 4b

If the my_stack in Question 4a is implemented by an array-based list in Python, and the first element of my_q has the memory address of 2345, what is the memory address of the last element in my_stack if each memory address occupies one byte? Justify your answer.

(5 marks)

Question 4c

If the my_stack in Question 4a is implemented by an array-based list in Python, what is the size of the underlying dynamic array if six more push operations are performed? Justify your answer.

(5 marks)

Question 5a

Complete the code below to implement an algorithm in Python to insert a new node with element e before a given successor in a doubly linked list called myDLL. The myDLL object has references to the head and tail of the linked list. Note that each node object has a next reference, a prev reference and an element reference.

```
def insert_node (myDLL, successor, e):
    new_node = Node(e) # create the new node
    # complete code here

    return new_node
```

(7 marks)

Question 5b

Complete the code below to implement an algorithm in Python to insert a new node with element e before a given successor in a singly linked list called mySLL. The mySLL object has references to the head and tail of the linked list. Note that each node object has a next reference and an element reference.

```
def insert_node (mySLL, successor, e):
    new_node = Node(e) # create the new node
    # complete code here

    return new_node
```

(8 marks)

Question 6

Based on the tree shown below in Figure Q6, answer the following questions.

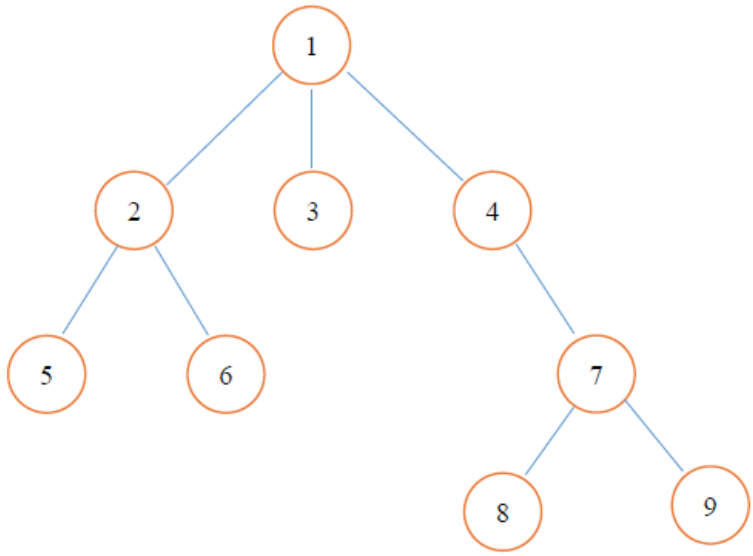

Figure Q6

Question 6a

How many external nodes in the tree?

(2 marks)

Question 6b

What is the degree of node 3?

(2 marks)

Question 6c

What is the depth of node 8?

(2 marks)

Question 6d

What is the height of node 4?

(2 marks)

Question 6e

What is the overall order of visits in postorder traversal? Analyze its time complexity.

(7 marks)

Question 7a

Complete the code below to implement an algorithm in Python to insert a node into an array-based list called myList at a given index. Do not use built-in methods in Python list, e.g., myList.insert(index, node).

```
def insert_node (myList, node, index):
    # complete code here
    # myList.size denotes the current size of myList
    # inserting an element requires incrementing the size by one

    return myList
```

(10 marks)

Question 7b

Analyze the time complexity of the algorithm in Question 7a. Which data structure(s) is efficient for element removal at arbitrary positions?

(5 marks)

----- END OF PAPER -----

1(a) The order is 2^{12}, $5n + 2\log(n)$, $2n\log(n) + 4n$, $3n^2$

$2n\log(n) + 4n = O(n \log n)$
$2^{12} = O(1)$
$5n + 2\log(n) = O(n)$
$3n^2 = O(n^2)$

$O(1) < O(n) < O(n\log n) < O(n^2)$

(b) The for-loop executes n times where $n = num$.

Hence, the time complexity is $O(n)$

(c)
```
def find_factors(num):
    result = [ ]
    i = 1
    while i * i ≤ num:
        if num % i == 0:
            result.append(i)
            j = num / i
            if i != j:
                result.append(j)
        i = i+1
    return result
```

time complexity = $O(\sqrt{n})$

because the while-loop executes $\sqrt{n}$ times where $n = num$

2.
```
def isPalindrome(word: str) -> bool:
    if len(word) == 0:
        return True
    i = 0
    j = len(word) - 1
    while i < j:
        if word[i] != word[j]:
            return False
        i = i+1
        j = j-1
    return True
```

3(a) Stack, because Last-In-First-Out property is required for retrieving addresses.

(b) Queue, because First-In-First-Out property is required for retrieving orders.

(c) [[0,1,1], [2,2,2], [3,3,3]]

(d) The bug is that the copy operation was a shallow copy instead of a deep copy.

(d) replace list2 = list1.copy() with

```
def deepcopy(L):
    result = []
    for item in L:
        if isinstance(item, list):
            result.append(deepcopy(item))
        else:
            result.append(item)
    return result

list2 = deepcopy(list1)
```

4(a)

operation	response	Elements
my-stack.push(10)	True	10
my-stack.push(12)	True	10, 12
my-stack.pop()	12	10
my_stack.size()	1	10
my_stack.push(5)	True	10, 5
my-stack.pop()	5	10
my-stack.push(14)	True	10, 14
my-stack.push(10)	True	10, 14, 10

(b) memory address of last element

$= 2345 + (3-1)$ [there are 3 elements in the stack]

$= 2347$

because each element occupies 1 byte of memory

(c) assuming the array is initialised at capacity of 5, pushing the 3rd element on the stack would trigger the resize operation that doubles the array capacity from 5 to 10. After all 6 pushes, the size of the stack is 9 which is still less than its capacity.

$\therefore$ size of underlying array = capacity of stack

$= 10$.

5(a)

```
def insertnode(myDLL, successor, e):

    new_node = Node(e)
    # assumes myDLL is implemented with head as a sentinel node
    if successor is None:
        # append new node to the linked list.
        myDLL.tail.next = new-node
        new_node.prev = myDLL.tail.
        myDLL.tail = new-node
    else:

        prev = successor.prev

        prev.next = new_node
        new_node.prev = prev

        new_node.next = successor
        successor.prev = new_node.
```

(b)

```
def insert_node(mySLL, successor, e):
    new_node = Node(e)

    if successor == mySLL.head
        new_node.next = mySLL.head
        mySLL.head = new_node
    elif successor is None:
        mySLL.tail.next = new_node
        mySLL.tail = new_node
    else:
        current = mySLL.head
        while current.next != successor:
            current = current.next
        current.next = new-node
        new_node.next = successor
```

6(a) external nodes = {3, 5, 6, 8, 9}

because these nodes don't have children

(b) degree of node 3 = 0

because node 3 has no children

(c) depth of node 8 = 3

because node 8 has 3 ancestors

(d) height of node 4 = 2

because the max depth of its descendents is 2.

(e) overall order is 5, 6, 2, 3, 8, 9, 7, 4, 1

time complexity = $n \times O(1)$

$= O(n)$

where there are n nodes

time complexity of each visit is $O(1)$

7(a)

```
def insert_node(myList, node, index):
    if index < 0 or index > myList.size
        raise IndexError("invalid index number").
    i = myList.size
    while i > index
        myList[i] = myList[i-1]
        i -= 1
    myList[index] = node
    myList.size += 1
    return myList.
```

← assumes linked list is based on a fixed array of infinite capacity

assume fixed array shifting elements

(b) worst case happens when inserting at index 0

time complexity = 2 comparisons + 1 assignment + (n+1) comparisons +
$\underbrace{n}_{\text{no. of while loops}}$ × (2 array access + 2 subtraction + 2 assignments)
+ 1 array access + 1 addition + 1 assignment + 1 return

$= 5 + n + 6n$
$= 7n + 5$
$= O(n)$

Positional List is an efficient data structure because the time complexity for removing elements at arbitary position is $O(1)$.